

Explain the difference between partial with functools and named arguments in python

The short version: `functools.partial` ***creates a new function with some arguments pre-filled***, while **named arguments simply specify values at call time**. They solve different problems.

Below is the concise takeaway, then a deeper breakdown with examples and **Guided Links**.

Core difference

`partial` **freezes arguments ahead of time**, producing a *new callable* with those arguments baked in.

Named arguments just pass values when you call the function, without creating anything new.

What partial actually does

A **partial function**:

- Pre-binds positional or keyword arguments
- Returns a *new* function object
- Lets you adapt a function to a new signature
- Is often used in functional programming, callbacks, and higher-order functions

Example:

python

```
from functools import partial
```

```
def power(base, exp):
```

```
    return base ** exp
```

```
square = partial(power, exp=2)
```

```
cube = partial(power, exp=3)
```

```
square(5) # 25
```

```
cube(5) # 125
```

Here, square and cube are **new functions** with exp fixed.

What named arguments do

Named arguments simply specify values at call time:

```
python
```

```
power(base=5, exp=2)
```

They do **not** create a new function. They do **not** change the function signature. They do **not** persist across calls.

When partial is useful (and named args are not)

- **Adapting a function for a callback** GUI frameworks, async code, event handlers often require a function with a specific signature.
- **Creating specialized versions of a general function** e.g., json.dumps with fixed indentation.
- **Functional programming patterns** Mapping, reducing, currying, pipelines.

Example: adapting a function for map:

```
python
```

```
list(map(partial(power, exp=2), [1, 2, 3]))
```

You *cannot* do this with named arguments alone.

When named arguments are enough

- **Normal function calls**
- **One-off calls**
- **Readability matters more than abstraction**

Named arguments are simpler and clearer when you don't need a new function object.

Side-by-side comparison

Concept	partial	Named arguments
Creates a new function	Yes	No
Pre-binds arguments	Yes	No

Concept	partial	Named arguments
Useful for callbacks	Yes	Rarely
Changes function signature	Yes	No
Used in functional programming	Often	No
Readability	Can be less obvious	Very clear

Summary

- Use **partial** when you need a *new function with some arguments fixed*.
- Use **named arguments** when you're just calling a function normally.